

Arduino alapú automatikus kutyatáp adagoló

Digitális Technika II. Beadandó Feladat

Készítette: Dávid Szabolcs (PYOGHD)

Ismertető

A projekt célja egy automatizált kutyatáp adagológép elméleti megtervezése és szimulációja. Kutyatartás során gyakori probléma, hogy a gazdik elutazásakor gondot okoz a kutya megettetése, ráadásul megtartva a rendszeres időpontot. Bár a piacon számos (főleg macskáknak szánt) időzített etető létezik, de a nagyobb testű kutyák esetén, ezek nem lennének hosszú életűek, figyelembe véve a garatban maradt következő napi táp csábítását.

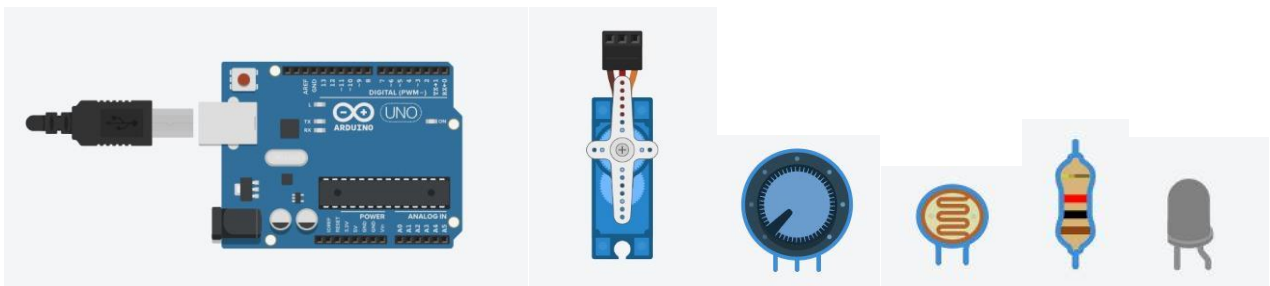
A projekt a Digitális Technika 2 tantárgy keretein belül készült, fókuszálva az Arduino mikrokontrolleres vezérlésre, a környezeti szenzorok (fényérzékelés) integrálására, valamint a valós idejű működés szimulációs környezetben (Tinkercad) történő modellezésére.

Tartalomjegyzék

Ismertető.....	1
Felhasznált alkatrészecskék.....	2
Megvalósítás és Áramköri terv	2
Szoftveres megvalósítás: 1. Verzió.....	3
Szoftveres megvalósítás: 2. Verzió.....	5
Felmerült hibák, korlátok és továbbfejlesztési lehetőségek.....	8

Alkatrész lista

Név	Mennyiség	Összetevő
U1	1	Arduino Uno R3
SERVO1	1	Pozicionális Mikroszervo
RFényérzékelő	1	Fotoellenállás
RpotIdőzítő	1	250 kΩ Szabályozható ellenállás
DVilágítás	1	Zöld LED
RLED ellenállás	1	220 Ω Ellenállás
RFotoszenzor ellenállás	1	10 kΩ Ellenállás



A rendszer egy központi feldolgozóegységből (Arduino UNO R3) és a hozzá kapcsolódó perifériákból áll:

- **Bemenetek:** Analóg fényérzékelő (LDR) a környezeti fényviszonyok monitorozására, valamint egy potenciométer a napi 24 órás ciklus szimulálására a tesztkörnyezetben.
- **Kimenetek:** Egy SG90 mikroszervó az adagolónyílás nyitására és zárására, valamint egy LED világítás, amely sötétedés utáni etetés esetén biztosít helyi megvilágítást.

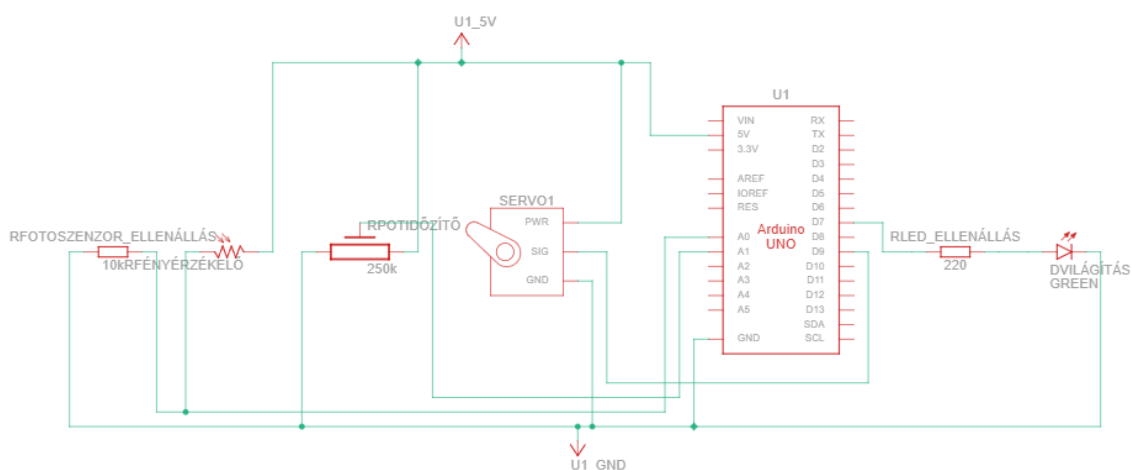
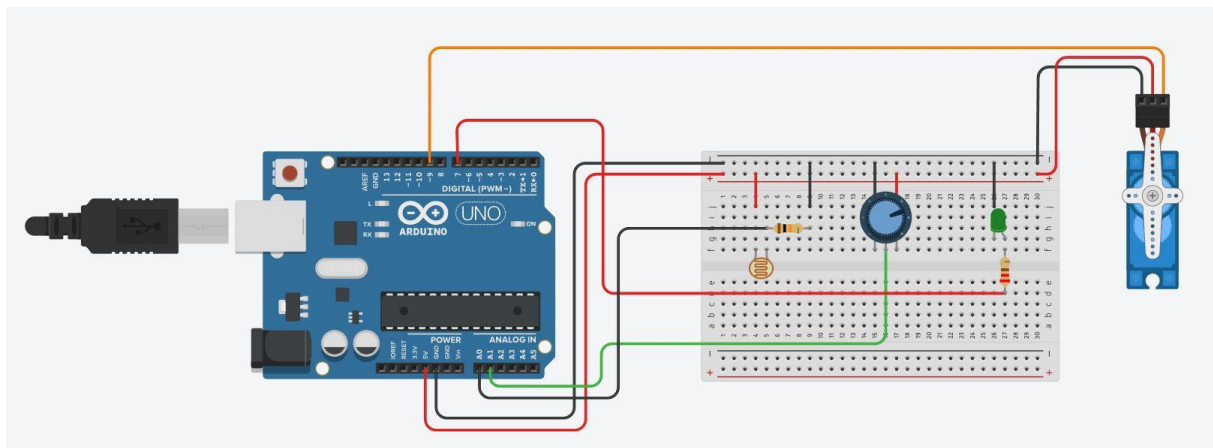
Megvalósítás és Áramköri terv

A prototípus áramköri kialakítása a következő alkatrészeket tartalmazza:

- **Arduino UNO R3**
- **SG90 Mikroszervó** (D9-es PWM lábra csatlakoztatva)
- **Fényérzékelő (LDR)** feszültségosztó kapcsolásban egy **10 kΩ-os ellenállással** (A0 analóg lábra csatlakoztatva)

- **Fehér LED** dióda egy **220 Ω -os áramkorlátozó ellenállással** (D7 digitális lábra csatlakoztatva)
- **10 k Ω Potenciométer** (A1 analóg lábra csatlakoztatva a szimulációhoz)
- Próbapanel és vezetékek.

A feszültségosztó jelentősége: Az LDR ellenállása a rá eső fény függvényében változik. Mivel az Arduino analóg bemenetei feszültséget mérnek, az LDR-t egy fix 10 k Ω -os ellenállással sorba kellett kötni. A két ellenállás közös csomópontjának potenciálja a környezeti fénytől függően változik, ezt a feszültség szintet olvassa be az A0-s port.



Szoftveres megvalósítás: 1. Verzió (Szimulációs modell)

A Tinkercad környezet sajátosságai miatt a 24 órás időpörgetést egy potenciométer analóg jelének leképezésével (map() függvény) szimuláltam. Amikor az idő eléri a beállított értéket (19:00), a rendszer kinyitja az adagolót pontosan 2 másodpercre. Ha a szenzor sötétet érzékel, a világítás bekapcsol, és a motor leállása után is aktív marad további 118 másodpercig (így jön ki a 2 perces világítás). A dupla etetés elkerülését egy logikai változó (alreadyFed) biztosítja.

A szimulációs projekt C++ kódja:

```
#include <Servo.h>

Servo dogFeeder;

int ldrPin = A0;    // Fényérzékelő
int timePin = A1;   // Potenciométer (Óra szimulátor)
int ledPin = 7;     // Éjszakai fény (LED)
int servoPin = 9;   // Adagoló motor
int lightThreshold = 400; // Sötétség határa
int feedingHour = 19; // Esti etetés ideje (19:00)

bool alreadyFed = false;

void setup() {
    Serial.begin(9600);
    dogFeeder.attach(servoPin);
    dogFeeder.write(0);
    pinMode(ledPin, OUTPUT);
}

void loop() {
    int timeVal = analogRead(timePin);
    int currentHour = map(timeVal, 0, 1023, 0, 24);
    int lightLevel = analogRead(ldrPin);

    if (currentHour == feedingHour) {
        if (!alreadyFed) {
            Serial.print("Etetes indul! Szimulalt ora: ");
            Serial.println(currentHour);

            if (lightLevel < lightThreshold) {
                digitalWrite(ledPin, HIGH);
            }
        }
    }
}
```

```
dogFeeder.write(90);  
delay(2000);  
  
dogFeeder.write(0);  
delay(118000);  
  
digitalWrite(ledPin, LOW);  
alreadyFed = true;  
}  
} else {  
    alreadyFed = false;  
}  
delay(100);  
}
```

Szoftveres megvalósítás: 2. Verzió (Fizikai prototípus modell - Belső időzítővel)

A fizikai megvalósítás során a potenciométer kikerül a rendszerből. Egy egyszerűsített változat esetén a mikrokontroller beépített óráját (millis() függvény) használjuk. A rendszer a bekapcsolás (áram alá helyezés) pillanatában elvégzi az első etetést, majd onnantól kezdve szigorúan 24 óránként ismétli a ciklust.

A belső óras ciklus C++ kódja:

```
#include <Servo.h>

Servo dogFeeder;

int ldrPin = A0;
int ledPin = 7;
int servoPin = 9;

int lightThreshold = 400;

unsigned long previousMillis = 0;
unsigned long interval = 86400000; // 24 óra ms-ban

bool firstFeedDone = false;

void setup() {
  Serial.begin(9600);
  dogFeeder.attach(servoPin);
  dogFeeder.write(0);
  pinMode(ledPin, OUTPUT);
}

void feedDog() {
  Serial.println("Valos ideju etetes indul!");
  int lightLevel = analogRead(ldrPin);

  if (lightLevel < lightThreshold) {
    digitalWrite(ledPin, HIGH);
  }

  dogFeeder.write(90);
  delay(2000);
}
```

```
dogFeeder.write(90);  
delay(2000);  
dogFeeder.write(0);  
delay(118000);  
digitalWrite(ledPin, LOW);  
}  
  
void loop() {  
    unsigned long currentMillis = millis();  
  
    if (!firstFeedDone) {  
        feedDog();  
        previousMillis = millis();  
        firstFeedDone = true;  
    }  
  
    if (currentMillis - previousMillis >= interval) {  
        previousMillis = currentMillis;  
        feedDog();  
    }  
}
```

Felmerült hibák, korlátok és továbbfejlesztési lehetőségek

1. A belső időzítő hálózathatófüggősége: A 2. Verzióban bemutatott millis() függvényes megoldás bár szoftveresen működik, fizikai környezetben egy gyenge ponttal rendelkezik. Egy esetleges áramszünet esetén az Arduino memóriája törlődik és a 24 órás ciklus nullázódik, ami az etetési ritmus elcsúszásához vezet.

2. Továbbfejlesztés: RTC modul integrálása: Az áramszünet okozta adatvesztés kiküszöbölésére a végleges megvalósításba egy **DS1307 (vagy DS3231) Valós idejű óra (RTC)** modult kell beépíteni. Ez az alkatrész I2C buszon (SDA és SCL lábakon) kommunikál az Arduinóval és egy CR2032-es gombelem segítségével a hálózati áramellátás megszűnése esetén is évekig pontosan méri az időt. Mivel a Tinkercad alapértelmezett eszközkészletében ez az alkatrész nem elérhető, a digitális prototípusban a szimulált modellek kerültek bemutatásra.

3. Későbbi fejlesztési irányok: A rendszer bővíthető lenne a táptartály alá épített mérőcellával, amely nemcsak az időt, hanem az adagolt táp pontos gramm/tömeg értékét is monitorozná a még precízebb etetés érdekében. Sajnos ez a kutya természete miatt nem megvalósítható, mivel adagoláskor nem valószínű, hogy kivárná a ciklusidőt és másnapra a tál sem lenne a helyén.

4. Mikroszervo hatékonysága: Az adagolóegység kialakításakor felmerül, hogy ha a szervo nem elég erős a tápadagoló elzárásához, akkor helyette relés megoldással egy erősebb léptetőmotor látná el ezt a feladatot vagy egy MG996R modellel.

5. A manuális nyomógomb lehetőségét biztonsági okokból (a kutya tanulékonyága és a túletetés veszélye miatt) a tervezési fázisban elvetettem.

